



Evolutionary Computation

Assignment 6: MSLS and ILS

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktoria (156070)



Contents

1	Introduction	1
2	Pseudocode	2
3	Plots	5
4	Results	10
5	Conclusion	10
6	All_Results	11

1 Introduction

The purpose of this report is to explore advanced metaheuristics that overcome the limitations of single-run local search algorithms. We implement and compare two complementary metaheuristic approaches: Multiple Start Local Search (MSLS) and Iterated Local Search (ILS).

We implemented two metaheuristics using basic steepest local search with 2-opt as the core improvement procedure: Multiple Start Local Search (MSLS) and Iterated Local Search (ILS).

MSLS performs 20 independent runs, each executing 200 iterations of local search from random starting solutions. The best solution found across all iterations within a run is retained. This approach explores the solution space broadly with no information sharing between runs.

ILS alternates between local search and perturbation phases. Starting from a random solution improved by local search, it iteratively perturbs the current best solution, applies local search, and accepts the result only if it strictly improves the objective.

The perturbation strategy : four distinct cut points are randomly selected in the tour, dividing it into four segments A, B, C, and D. These segments are reconnected as **A-C-B-D** instead of A-B-C-D. This removes four edges and creates four new ones, generating substantial structural changes while preserving tour validity.

We perform 20 ILS runs per instance, each starting from a different random solution. The stopping condition is the average execution time of MSLS for fair comparison. We report the number of local search executions per run to analyze the exploration-computation trade-off.

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory

2 Pseudocode

In this section, we will present the pseudocodes of the algorithm. For this assignment, we will code in C++.

Multiple Start Local Search (MSLS)

```
FUNCTION MSLS(dist_matrix, nodes, num_runs, iterations_per_run):  
    best_global_solution  $\leftarrow$  NULL  
    best_global_score  $\leftarrow$   $\infty$   
  
    FOR run FROM 1 TO num_runs DO :  
        best_run_solution  $\leftarrow$  NULL  
        best_run_score  $\leftarrow$   $\infty$   
  
        FOR iter FROM 1 TO iterations_per_run DO :  
            selected_nodes  $\leftarrow$  SELECT_NODES(nodes)  
            initial_solution  $\leftarrow$  RANDOM_SOLUTION(selected_nodes)  
  
            improved_solution  $\leftarrow$  STEEPEST_LOCAL_SEARCH(initial_solution,  
dist_matrix, "edge", nodes)  
            score  $\leftarrow$  COMPUTE_OBJECTIVE(improved_solution, dist_matrix,  
nodes)  
  
            IF score < best_run_score THEN :  
                best_run_score  $\leftarrow$  score  
                best_run_solution  $\leftarrow$  improved_solution  
  
            IF best_run_score < best_global_score THEN :  
                best_global_score  $\leftarrow$  best_run_score  
                best_global_solution  $\leftarrow$  best_run_solution  
  
    RETURN best_global_solution, best_global_score
```


Iterated Local Search (ILS)

```
FUNCTION ILS(dist_matrix, nodes, num_runs, time_limit):  
  best_global_solution  $\leftarrow$  NULL  
  best_global_score  $\leftarrow \infty$   
  
  FOR run FROM 1 TO num_runs DO :  
    selected_nodes  $\leftarrow$  SELECT_NODES(nodes)  
    x  $\leftarrow$  RANDOM_SOLUTION(selected_nodes)  
    x  $\leftarrow$  STEEPEST_LOCAL_SEARCH(x, dist_matrix, "edge", nodes)  
  
    best_solution  $\leftarrow$  x  
    best_score  $\leftarrow$  COMPUTE_OBJECTIVE(x, dist_matrix, nodes)  
    local_search_count  $\leftarrow$  1  
    start_time  $\leftarrow$  CURRENT_TIME()  
  
    WHILE (CURRENT_TIME() - start_time) < time_limit DO :  
      y  $\leftarrow$  PERTURBATION(x)  
      y  $\leftarrow$  STEEPEST_LOCAL_SEARCH(y, dist_matrix, "edge", nodes)  
      local_search_count  $\leftarrow$  local_search_count + 1  
  
      score  $\leftarrow$  COMPUTE_OBJECTIVE(y, dist_matrix, nodes)  
  
      IF score < best_score THEN :  
        x  $\leftarrow$  y  
        best_score  $\leftarrow$  score  
        best_solution  $\leftarrow$  y  
  
      IF best_score < best_global_score THEN :  
        best_global_score  $\leftarrow$  best_score  
        best_global_solution  $\leftarrow$  best_solution  
  
  RETURN best_global_solution, best_global_score
```

Perturbation

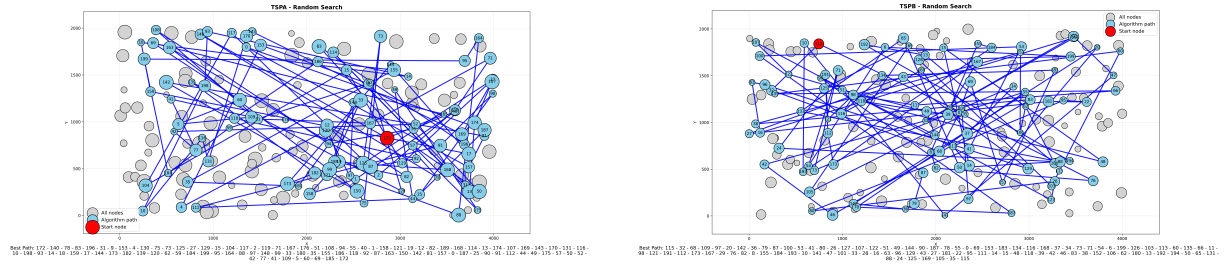
```
FUNCTION PERTURBATION(path):  
  n ← LENGTH(path)  
  IF n < 8 THEN : RETURN path  
  
  cut_set ← EMPTY_SET  
  WHILE SIZE(cut_set) < 4 DO :  
    random_position ← RANDOM(1, n-2)  
    ADD random_position TO cut_set  
  
  cuts ← SORTED_LIST(cut_set)  
  
  segment_A ← path[0 : cuts[0])  
  segment_B ← path[cuts[0] : cuts[1])  
  segment_C ← path[cuts[1] : cuts[2])  
  segment_D ← path[cuts[2] : n)  
  
  perturbed_path ← CONCATENATE(segment_A, segment_C, segment_B,  
    segment_D)  
  
  RETURN perturbed_path
```

3 Plots

After trying this algorithm, we will check its effectiveness by showing the constructed cycle.

In the plot, the cost of each node is represented by its size: larger nodes indicate a higher cost, while smaller ones indicate a lower cost. Additionally, all possible nodes are displayed, but the nodes chosen for the path are shown in blue, with their IDs printed on them. The started node is pointed with red color.

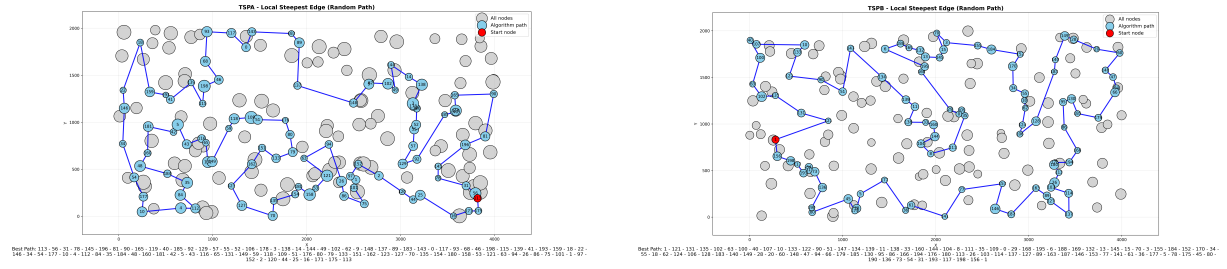
Figures for Steepest Local Search for Random Solution



(a) TSPA

(b) TSPB

Figure 1: Local Steepest Node (Random Path)



(a) TSPA

(b) TSPB

Figure 2: Local Steepest Node (Random Path)

Figures for Steepest Local Search with Candidate Moves

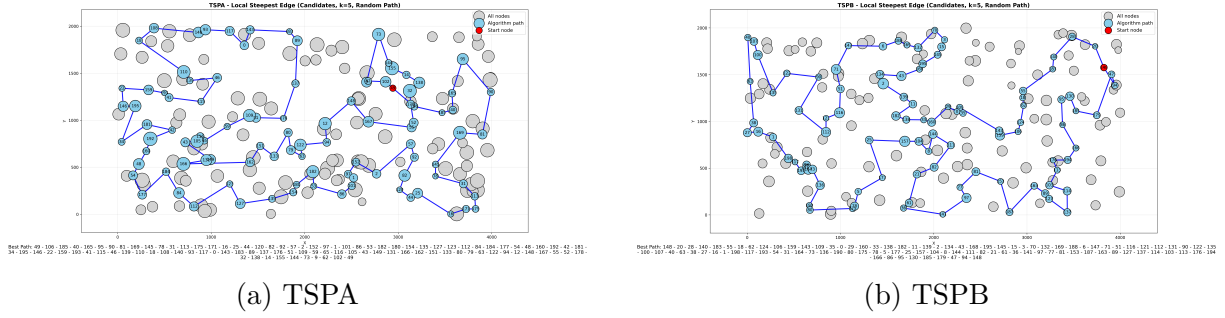


Figure 3: Local Steepest Node (Random Path)

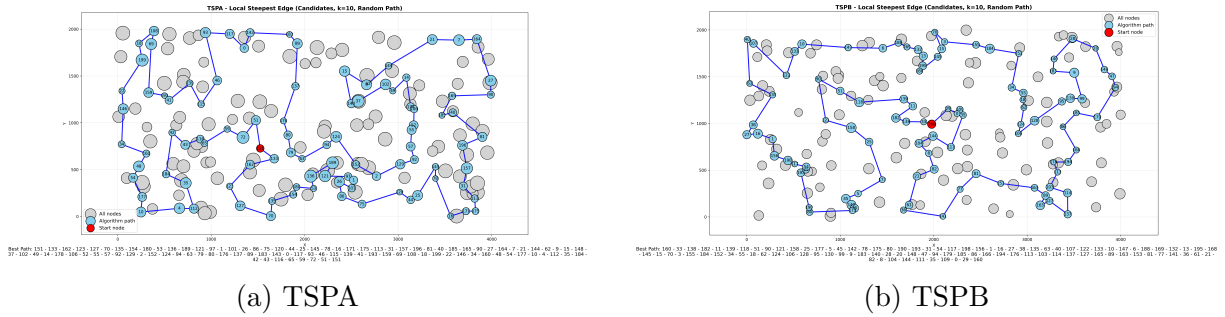


Figure 4: Local Steepest Node (Random Path)

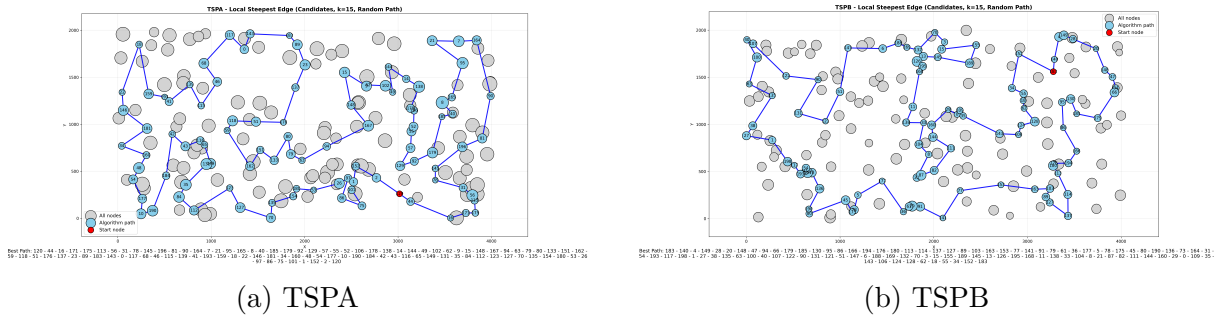


Figure 5: Local Steepest Node (Random Path)

Figures for Steepest Local Search with LM

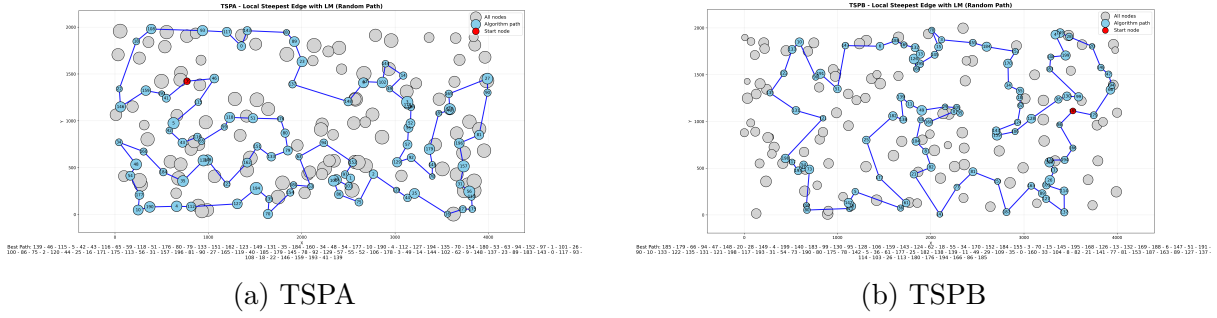


Figure 6: Local Steepest Node (Random Path)

Figures for Multiple Start Local Search (MSLS)

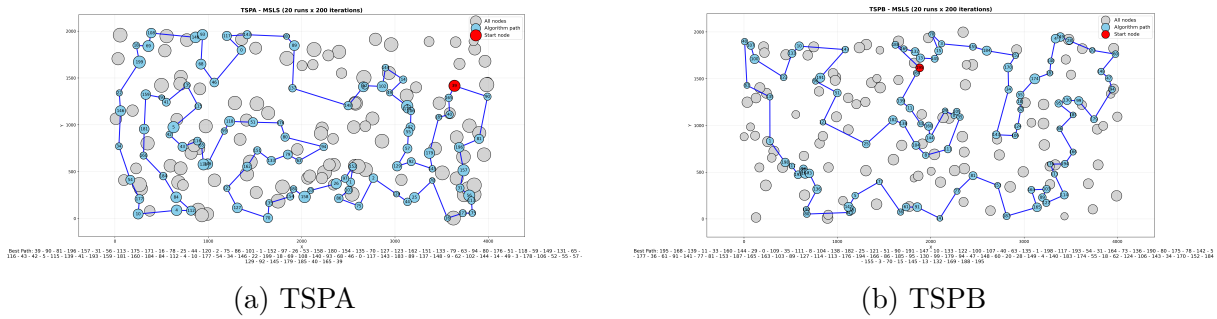


Figure 7: Multiple Start Local Search (MSLS)

Figures for Iterated local search (ILS)

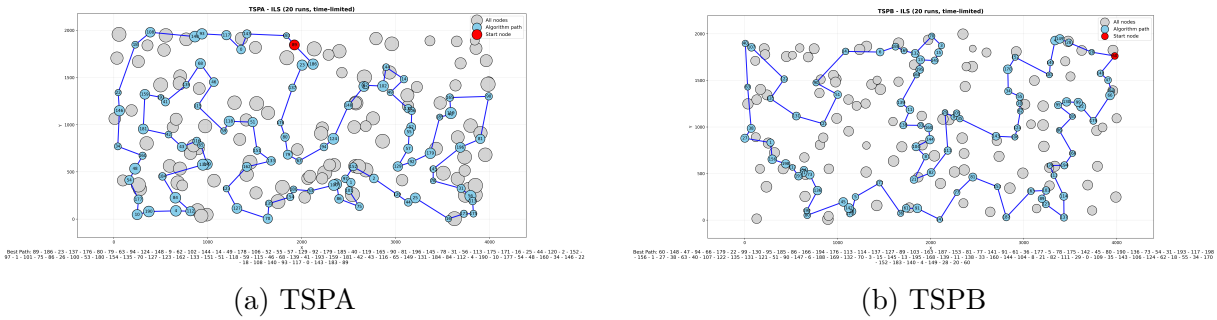


Figure 8: Iterated Local Search (ILS)

List of Nodes of Best Results for TSPA
Local Steepest Edge (Random Path)
113 - 56 - 31 - 78 - 145 - 196 - 81 - 90 - 165 - 119 - 40 - 185 - 92 - 129 - 57 - 55 - 52 - 106 - 178 - 3 - 138 - 14 - 144 - 49 - 102 - 62 - 9 - 148 - 137 - 89 - 183 - 143 - 0 - 117 - 93 - 68 - 46 - 198 - 115 - 139 - 41 - 193 - 159 - 18 - 22 - 146 - 34 - 54 - 177 - 10 - 4 - 112 - 84 - 35 - 184 - 48 - 160 - 181 - 42 - 5 - 43 - 116 - 65 - 131 - 149 - 59 - 118 - 109 - 51 - 176 - 80 - 79 - 133 - 151 - 162 - 123 - 127 - 70 - 135 - 154 - 180 - 158 - 53 - 121 - 63 - 94 - 26 - 86 - 75 - 101 - 1 - 97 - 152 - 2 - 120 - 44 - 25 - 16 - 171 - 175 - 113
Local Steepest Edge with LM (Random Path)
139 - 46 - 115 - 5 - 42 - 43 - 116 - 65 - 59 - 118 - 51 - 176 - 80 - 79 - 133 - 151 - 162 - 123 - 149 - 131 - 35 - 184 - 160 - 34 - 48 - 54 - 177 - 10 - 190 - 4 - 112 - 127 - 194 - 135 - 70 - 154 - 180 - 53 - 63 - 94 - 152 - 97 - 1 - 101 - 26 - 100 - 86 - 75 - 2 - 120 - 44 - 25 - 16 - 171 - 175 - 113 - 56 - 31 - 157 - 196 - 81 - 90 - 27 - 165 - 119 - 40 - 185 - 179 - 145 - 78 - 92 - 129 - 57 - 55 - 52 - 106 - 178 - 3 - 49 - 14 - 144 - 102 - 62 - 9 - 148 - 137 - 23 - 89 - 183 - 143 - 0 - 117 - 93 - 108 - 18 - 22 - 146 - 159 - 193 - 41 - 139
MSLS (20 runs x 200 iterations)
39 - 90 - 81 - 196 - 157 - 31 - 56 - 113 - 175 - 171 - 16 - 78 - 25 - 44 - 120 - 2 - 75 - 86 - 101 - 1 - 152 - 97 - 26 - 53 - 158 - 180 - 154 - 135 - 70 - 127 - 123 - 162 - 151 - 133 - 79 - 63 - 94 - 80 - 176 - 51 - 118 - 59 - 149 - 131 - 65 - 116 - 43 - 42 - 5 - 115 - 139 - 41 - 193 - 159 - 181 - 160 - 184 - 84 - 112 - 4 - 10 - 177 - 54 - 34 - 146 - 22 - 199 - 18 - 69 - 108 - 140 - 93 - 68 - 46 - 0 - 117 - 143 - 183 - 89 - 137 - 148 - 9 - 62 - 102 - 144 - 14 - 49 - 3 - 178 - 106 - 52 - 55 - 57 - 129 - 92 - 145 - 179 - 185 - 40 - 165 - 39
ILS (20 runs, time-limited)
89 - 186 - 23 - 137 - 176 - 80 - 79 - 63 - 94 - 124 - 148 - 9 - 62 - 102 - 144 - 14 - 49 - 178 - 106 - 52 - 55 - 57 - 129 - 92 - 179 - 185 - 40 - 119 - 165 - 90 - 81 - 196 - 145 - 78 - 31 - 56 - 113 - 175 - 171 - 16 - 25 - 44 - 120 - 2 - 152 - 97 - 1 - 101 - 75 - 86 - 26 - 100 - 53 - 180 - 154 - 135 - 70 - 127 - 123 - 162 - 133 - 151 - 51 - 118 - 59 - 115 - 46 - 68 - 139 - 41 - 193 - 159 - 181 - 42 - 43 - 116 - 65 - 149 - 131 - 184 - 84 - 112 - 4 - 190 - 10 - 177 - 54 - 48 - 160 - 34 - 146 - 22 - 18 - 108 - 140 - 93 - 117 - 0 - 143 - 183 - 89
Local Steepest Edge Candidates(k=10, RandomPath)
151 - 133 - 162 - 123 - 127 - 70 - 135 - 154 - 180 - 53 - 136 - 189 - 121 - 97 - 1 - 101 - 26 - 86 - 75 - 120 - 44 - 25 - 145 - 78 - 16 - 171 - 175 - 113 - 31 - 157 - 196 - 81 - 40 - 185 - 165 - 90 - 27 - 164 - 7 - 21 - 144 - 62 - 9 - 15 - 148 - 37 - 102 - 49 - 14 - 178 - 106 - 52 - 55 - 57 - 92 - 129 - 2 - 152 - 124 - 94 - 63 - 79 - 80 - 176 - 137 - 89 - 183 - 143 - 0 - 117 - 93 - 46 - 115 - 139 - 41 - 193 - 159 - 69 - 108 - 18 - 199 - 22 - 146 - 34 - 160 - 48 - 54 - 177 - 10 - 4 - 112 - 35 - 184 - 42 - 43 - 116 - 65 - 59 - 72 - 51 - 151

Table 1: Best paths for each algorithm TSPA

List of Nodes of Best Results for TSPB
Local Steepest Edge (Random Path)
1 - 121 - 131 - 135 - 102 - 63 - 100 - 40 - 107 - 10 - 133 - 122 - 90 - 51 - 147 - 134 - 139 - 11 - 138 - 33 - 160 - 144 - 104 - 8 - 111 - 35 - 109 - 0 - 29 - 168 - 195 - 6 - 188 - 169 - 132 - 13 - 145 - 15 - 70 - 3 - 155 - 184 - 152 - 170 - 34 - 55 - 18 - 62 - 124 - 106 - 128 - 183 - 140 - 149 - 28 - 20 - 60 - 148 - 47 - 94 - 66 - 179 - 185 - 130 - 95 - 86 - 166 - 194 - 176 - 180 - 113 - 26 - 103 - 114 - 137 - 127 - 89 - 163 - 187 - 146 - 153 - 77 - 141 - 61 - 36 - 177 - 5 - 78 - 175 - 45 - 80 - 190 - 136 - 73 - 54 - 31 - 193 - 117 - 198 - 156 - 1
Local Steepest Edge with LM (Random Path)
185 - 179 - 66 - 94 - 47 - 148 - 20 - 28 - 149 - 4 - 199 - 140 - 183 - 99 - 130 - 95 - 128 - 106 - 159 - 143 - 124 - 62 - 18 - 55 - 34 - 170 - 152 - 184 - 155 - 3 - 70 - 15 - 145 - 195 - 168 - 126 - 13 - 132 - 169 - 188 - 6 - 147 - 51 - 191 - 90 - 10 - 133 - 122 - 135 - 131 - 121 - 198 - 117 - 193 - 31 - 54 - 73 - 190 - 80 - 175 - 78 - 142 - 5 - 36 - 61 - 177 - 25 - 182 - 138 - 139 - 11 - 49 - 29 - 109 - 35 - 0 - 160 - 33 - 104 - 8 - 82 - 21 - 141 - 77 - 81 - 153 - 187 - 163 - 89 - 127 - 137 - 114 - 103 - 26 - 113 - 180 - 176 - 194 - 166 - 86 - 185
MSLS (20 runs x 200 iterations)
195 - 168 - 139 - 11 - 33 - 160 - 144 - 29 - 0 - 109 - 35 - 111 - 8 - 104 - 138 - 182 - 25 - 121 - 51 - 90 - 191 - 147 - 10 - 133 - 122 - 100 - 107 - 40 - 63 - 135 - 1 - 198 - 117 - 193 - 54 - 31 - 164 - 73 - 136 - 190 - 80 - 175 - 78 - 142 - 5 - 177 - 36 - 61 - 91 - 141 - 77 - 81 - 153 - 187 - 165 - 163 - 103 - 89 - 127 - 114 - 113 - 176 - 194 - 166 - 86 - 185 - 95 - 130 - 99 - 179 - 94 - 47 - 148 - 60 - 20 - 28 - 149 - 4 - 140 - 183 - 174 - 55 - 18 - 62 - 124 - 106 - 143 - 34 - 170 - 152 - 184 - 155 - 3 - 70 - 15 - 145 - 13 - 132 - 169 - 188 - 195
ILS (20 runs, time-limited)
60 - 148 - 47 - 94 - 66 - 179 - 22 - 99 - 130 - 95 - 185 - 86 - 166 - 194 - 176 - 113 - 114 - 137 - 127 - 89 - 103 - 163 - 187 - 153 - 81 - 77 - 141 - 91 - 61 - 36 - 177 - 5 - 78 - 175 - 142 - 45 - 80 - 190 - 136 - 73 - 54 - 31 - 193 - 117 - 198 - 156 - 1 - 27 - 38 - 63 - 40 - 107 - 122 - 135 - 131 - 121 - 51 - 90 - 147 - 6 - 188 - 169 - 132 - 70 - 3 - 15 - 145 - 13 - 195 - 168 - 139 - 11 - 138 - 33 - 160 - 144 - 104 - 8 - 21 - 82 - 111 - 29 - 0 - 109 - 35 - 143 - 106 - 124 - 62 - 18 - 55 - 34 - 170 - 152 - 183 - 140 - 4 - 149 - 28 - 20 - 60
Local Steepest Edge Candidates(k=10, RandomPath)
160 - 33 - 138 - 182 - 11 - 139 - 118 - 51 - 90 - 121 - 158 - 25 - 177 - 5 - 45 - 142 - 78 - 175 - 80 - 190 - 193 - 31 - 54 - 117 - 198 - 156 - 1 - 16 - 27 - 38 - 135 - 63 - 40 - 107 - 122 - 133 - 10 - 147 - 6 - 188 - 169 - 132 - 13 - 195 - 168 - 145 - 15 - 70 - 3 - 155 - 184 - 152 - 34 - 55 - 18 - 62 - 124 - 106 - 128 - 95 - 130 - 99 - 9 - 183 - 140 - 28 - 20 - 148 - 47 - 94 - 179 - 185 - 86 - 166 - 194 - 176 - 113 - 103 - 114 - 137 - 127 - 165 - 89 - 163 - 153 - 81 - 77 - 141 - 36 - 61 - 21 - 82 - 8 - 104 - 144 - 111 - 35 - 109 - 0 - 29 - 160

Table 2: Best paths for each algorithm TSPB

4 Results

Method	TSPA (Avg, Min, Max)	TSPB (Avg, Min, Max)
Random Path	264487.27 (234703, 301288)	214424.73 (194631, 243761)
LS (Random Path)	74166.34 (71458, 78661)	48345.22 (45620, 52170)
LS with LM (Random Path)	74216.18 (70763, 78864)	49020.86 (46363, 53097)
MSLS (20 runs x 200 iterations)	71413.00 (71041, 71834)	45855.90 (45317, 46321)
ILS (20 runs, time-limited)	69618.15 (69107, 70725)	43830.45 (43460, 44557)
LS (Candidates, k=5, Random Path)	85670.98 (79970, 91607)	50673.71 (47369, 55117)
LS (Candidates, k=10, Random Path)	78030.57 (72952, 83417)	48629.81 (45428, 51589)
LS (Candidates, k=15, Random Path)	75469.30 (71931, 81031)	48496.35 (45358, 52079)

Table 3: Comparison of TSPA and TSPB results with MSLS and ILS (Avg, Min, Max)

Method	TSPA Time (avg, min, max) [μs]	TSPB Time (avg, min, max) [μs]
Random Path	0.0226 (0.0209, 0.0361)	0.0251 (0.0214, 0.0497)
LS (Random Path)	106.5689 (94.0115, 159.8687)	127.1989 (93.0454, 182.4959)
LS with LM (Random Path)	50.6919 (35.5602, 69.0622)	63.7943 (42.1606, 102.9832)
MSLS (20 runs x 200 iterations)	22106.5766 (20271.8255, 24661.7562)	21672.4893 (20421.4810, 24695.6286)
ILS (20 runs, time-limited)	22109.2723 (22106.7761, 22115.1909)	21674.9387 (21672.8717, 21678.2780)
LS (Candidates, k=5, Random Path)	7.1014 (6.4651, 11.1023)	8.7795 (6.6938, 21.5580)
LS (Candidates, k=10, Random Path)	11.0378 (9.9194, 17.4050)	14.1925 (10.7767, 28.9982)
LS (Candidates, k=15, Random Path)	15.4697 (13.6906, 21.3452)	19.7004 (14.6174, 41.2415)

Table 4: Average, minimum, and maximum execution times for TSPA and TSPB with MSLS and ILS

Metric	TSPA	TSPB
Average Local Searches per ILS Run	5107.55	5253.45
Min Local Searches	4802	4652
Max Local Searches	5824	5753

Table 5: ILS local search counts for TSPA and TSPB

5 Conclusion

Based on the results in Tables 3, 4, and 5, the following observations can be made:

MSLS and ILS significantly outperform both the basic steepest local search and the LM-based variant in solution quality across both instances. ILS consistently achieves the best results, demonstrating that guided perturbation-based exploration is more effective than independent random restarts.

The computational cost is substantial: both metaheuristics require considerably more time than single-run local search. However, this investment consistently yields superior solutions. ILS performs several thousand local searches per run, demonstrating intensive exploration of the solution space while maintaining a coherent search trajectory.

6 All Results

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Path	264188.61 (238257, 293017)	213973.45 (184171, 238914)
Nearest Neighbor (End)	85108.51 (83182, 89433)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)	48694.25 (44609, 57283)
Greedy Cycle	72635.98 (71488, 74410)	51400.60 (49001, 57324)
Greedy 2-Regret	115400.24 (105692, 126860)	72712.85 (67809, 78406)
Greedy 2-Regret (w=0.5)	72136.15 (71108, 73395)	50934.60 (47144, 55700)
NN-Flex 2-Regret	117138.49 (108151, 124921)	74444.46 (69933, 80278)
NN-Flex 2-Regret (w=0.5)	72407.59 (70010, 75452)	47664.46 (44891, 55247)
Local Greedy Node (Greedy Cycle)	70687.00 (70687, 70687)	46443.67 (46328, 51512)
Local Greedy Edge (Greedy Cycle)	70618.84 (70571, 70663)	46089.89 (45824, 51482)
Local Steepest Node (Greedy Cycle)	70687.00 (70687, 70687)	46466.54 (46371, 51512)
Local Steepest Edge (Greedy Cycle)	70571.00 (70571, 70571)	45996.84 (45867, 51259)
Local Greedy Node (Random Path)	85217.85 (77570, 94277)	60178.87 (54006, 66885)
Local Greedy Edge (Random Path)	74232.03 (71777, 77750)	48192.19 (46109, 51934)
Local Steepest Node (Random Path)	85271.00 (85271, 85271)	63491.00 (63491, 63491)
Local Steepest Edge (Random Path)	72687.00 (72687, 72687)	49190.00 (49190, 49190)
Local Steepest (Candidates, k=5)	88563.00 (88563, 88563)	52093.00 (52093, 52093)
Local Steepest (Candidates, k=10)	83598.00 (83598, 83598)	51153.00 (51153, 51153)
Local Steepest (Candidates, k=15)	81136.00 (81136, 81136)	48854.00 (48854, 48854)
Local Steepest Edge with LM (Random Path)	74187.41 (71646, 78764)	49144.47 (45888, 53039)

Table 6: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The 3 lowest average values in each column are highlighted in bold.

Method	TSPA Time Avg (Min, Max)	TSPB Time Avg (Min, Max)
Random Path	0.0036 (0.0034, 0.0144)	0.0039 (0.0036, 0.0232)
Nearest Neighbor (End)	0.0694 (0.0463, 0.1093)	0.0685 (0.0470, 0.1282)
Nearest Neighbor (Flexible)	91.0993 (85.8761, 119.1184)	94.7285 (87.4900, 154.2259)
Greedy Cycle	87.6467 (82.1853, 135.8448)	90.6419 (84.6333, 153.7690)
Greedy 2-Regret	13.4090 (12.2387, 18.2542)	13.4612 (12.4574, 21.9434)
Greedy 2-Regret (w=0.5)	12.9063 (11.8320, 18.2641)	12.6926 (11.2819, 20.7204)
NN-Flex 2-Regret	13.8750 (12.7669, 18.4833)	13.9626 (12.9940, 21.1062)
NN-Flex 2-Regret (w=0.5)	13.5540 (12.4458, 19.2731)	13.6345 (12.6257, 21.8523)
Local Greedy Node (Greedy Cycle*)	1.2763 (0.8682, 2.4062)	1.8935 (1.1700, 5.2516)
Local Greedy Edge (Greedy Cycle*)	1.5676 (1.1047, 3.2752)	2.5081 (1.3700, 5.8678)
Local Steepest Node (Greedy Cycle*)	0.5265 (0.4719, 1.0174)	0.7846 (0.4909, 1.6765)
Local Steepest Edge (Greedy Cycle*)	0.5418 (0.4837, 1.0700)	1.0705 (0.6829, 2.2377)
Local Greedy Node (Random Path)	57.7227 (44.0444, 123.7979)	51.7444 (41.5894, 92.8947)
Local Greedy Edge (Random Path)	58.0691 (47.6349, 99.7285)	52.4021 (45.1037, 84.4763)
Local Steepest Node (Random Path)	10.9774 (10.0351, 18.7647)	13.6565 (12.4567, 26.2057)
Local Steepest Edge (Random Path)	9.1549 (8.5878, 12.7278)	9.0840 (8.4419, 18.6255)
Local Steepest (Candidates, k=5)	1.4490 (1.3910, 1.9627)	1.4250 (1.3467, 1.8863)
Local Steepest (Candidates, k=10)	1.8156 (1.7346, 2.3018)	1.7992 (1.7277, 2.4961)
Local Steepest (Candidates, k=15)	2.1838 (2.0927, 2.7651)	2.1374 (2.0609, 2.9937)
Local Steepest Edge with LM (Random Path)	50.6919 (35.5602, 69.0622)	53.7797 (39.3278, 92.4221)

Table 7: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)